

Direct sampling & plots

Every distribution in `m.dist` can be sampled **directly**, outside of a model, by passing `sample=True`. This is useful for prior predictive checks, simulating data, or quickly inspecting a distribution's shape.

```
from BayesForge import bf
m = bf()
# Draw 1000 samples from a standard normal
x = m.dist.normal(0, 1, shape=(1000,), sample=True)
```

What `sample=True` returns

By default (`to_jax=False`) a direct sample is returned as a **SampledData** object — a thin wrapper around a JAX array. It behaves like a JAX array everywhere (arithmetic, indexing, `jnp.*` functions, `jit/vmap/grad`, use as a distribution parameter), and its `repr` reports the `dtype` so you can see the kind and precision at a glance:

```
from BayesForge import bf
m = bf()
x = m.dist.normal(0, 1, shape=(3,), sample=True)
x          # SampledData([ 0.63, 0.29, -0.40], dtype=float64)
```

The extra thing a **SampledData** gives you over a raw array is a set of **built-in plotting methods**.

If you only want the raw JAX array (no wrapper, no plotting helpers), pass `to_jax=True`:

```
x = m.dist.normal(0, 1, shape=(1000,), sample=True, to_jax=True)  # plain jax array
```

Plotting a sampled array

Each plotting method is called directly on the sampled object. Most accept `interactive=True` (the default, **Plotly**) or `interactive=False` (**matplotlib** / **seaborn**):

```
s = m.dist.normal(0, 1, shape=(1000, 4), sample=True) # 2D: [n_samples, n_variables]

s.hist() # interactive Plotly histogram
s.violinplot(interactive=False) # static matplotlib
s.corr_heatmap()
```

The available methods depend on the **shape** of the sampled array. `### 1D — (n_samples,)`

2D — (n_samples, n_variables)

The most common case: several variables sampled together.

```
s = m.dist.normal(0, 1, shape=(1000, 4), sample=True)

s.hist() # one histogram trace per column
s.density() # KDE per column (Plotly only)
s.boxplot() # one box per column
s.violinplot() # violin + box + mean line per column
s.pairplot() # pairwise scatter (max_vars=5)
s.corr_heatmap() # annotated correlation matrix (digits=5)
s.traceplot() # MCMC-style line per column
s.autocorr() # autocorrelation per column
s.ridgeline() # stacked densities (category_labels=, offset=2)
s.surface_3d() # surface plot (Plotly only)
```

Time series — (n_samples, n_time)

Treat the second axis as time and summarise across draws.

```
s = m.dist.gaussian_random_walk(scale=0.5, num_steps=50, shape=(500,), sample=True)

s.timeseries(credible_interval=0.9) # posterior mean + credible band
```

3D — (n_samples, n_groups, n_times)

```
s.hist()          # 3D ridged histograms across groups / times
s.ridgeline()     # also accepts 3D input
```

Posterior predictive

Compare draws against observed data.

```
# s has shape [n_draws, n_observations]
s.ppc_plot(observed_data, n_samples_to_plot=50)  # Plotly only
```

3D scatter — (n_samples, >=3)

```
s.scatter3d()     # 3D point cloud of the first three columns
```

A non-plot helper

Alongside the plots, `SampledData` exposes the highest-density interval:

```
low, high = s.hdi(cred_mass=0.95)
```

Notes & gotchas

Note

- `density`, `surface_3d`, and `ppc_plot` are **Plotly-only** — passing `interactive=False` has no effect on them.
- The dimensional requirement is **enforced**: e.g. `corr_heatmap`, `boxplot`, `violinplot`, `pairplot`, `traceplot`, `scatter3d`, and `timeseries` raise a `ValueError` if the array is not the expected shape.
- Plotting methods live on the `SampledData` wrapper only. If you sampled with `to_jax=True` (or later called `.to_jax()`), convert back with `SampledData(arr)` to plot, or re-sample without `to_jax`.